MAGMA

DevOps & Cloud [SWE] — Class 15 Notes

LECTURE TITLE	Kubernetes Troubleshooting *(the transcript header just says "Kubernetes 8")*
DATE/TIME	21 September 2026, 4:15 PM
DURATION	120 minutes
INSTRUCTOR	Nensi Ravaliya
BIG IDEA	When Kubernetes breaks, don't guess — walk the same ladder every time: **get** → **describe** → **events** → **logs** → **exec** → **test** → **fix** → **verify**. The status tells you *what*; the events and logs tell you *why*.
SOURCES	⚠️ **The lecture audio was not captured** (the transcript holds only ~30 seconds of screen-share setup). These notes are therefore built from **her own repo folder for this session**, `Nency-Ravaliya/devops-heros` → `session-14-kubernetes-troubleshooting` (`01-kubectl-get` … `09-service-dns-troubleshooting`, `mini-project/`, `scenarios/`, `README.md`), plus the copy of that folder and the two extra YAML files in your own class folder.

WHAT'S INSIDE

0 ⚡ The 60-Second Version

1 🗓️ Logistics — The Class That Wasn't Recorded

2 🧭 The Troubleshooting Mindset

3 👀 `kubectl get` — "What Is Happening?"

4 🔬 `kubectl describe` — "Why Is It Happening?"

5 📜 `kubectl logs` — "What Is the Application Saying?"

6 🚪 `kubectl exec` — "Let Me Check From Inside"

7 📰 Events — Kubernetes' Activity Log

8 🔁 `CrashLoopBackOff` — The App Keeps Dying

9 📦 `ImagePullBackOff` — Kubernetes Can't Get the Image

10 ⏳ `Pending` Pods — No Node Will Take It

11 🔌 Service Troubleshooting — Labels → Selector → Endpoints

12 🌐 DNS Troubleshooting — Can the Pods Find Each Other by Name?

13 🧩 The Mini-Project — Troubleshooting Challenge

14 🔥 The Triage Gauntlet — Five Broken Pods at Once

15 📁 Your Own Files — A Review

16 📝 Cheat-Sheet

17 🗒️ Notes on the Transcript Itself

WATCH OUT

Read this first — the recording failed, so this PDF is built from her repo, not from the lecture.

The transcript for this class contains about **thirty seconds** of "*are you able to see my screen?*" and then stops. The two hours of teaching were never captured.

So, to be completely honest about what's in here:

- **No quotes.** There are no 🎤 "*said in class*" cards, because nothing she said was recorded.
- **No class moments.** No 😄 jokes, failures or banter — none were captured, and none have been invented.
- **No quiz tables.** If there was a team relay or a Q&A drill, it's lost.
- **Everything technical comes from her repo folder for this session**, which is the material she screen-shared. Every YAML file, command and expected output below is taken from those files. Where these notes add something of their own (an analogy, an explainer, a standard fact like back-off timings or exit codes), it is **labelled as filled in**.

Phrases like "*her repo says*" or "*her README's rule*" below mean exactly that — the written material, not the spoken lecture.

NOTE

Numbering, as usual, doesn't line up. In her repo this is `session-14-kubernetes-troubleshooting`; your folder is `Class-15`. The previous class (Class 14, 17 Sep) was ConfigMaps/Secrets/Ingress, the next one (Class 16, 22 Sep) is Helm, and **Class 21 (4 Oct) is "troubleshooting part 2"** — so this is **part 1**. Go by content, not by number.

NOTE

Two things in her repo that are *not* in your copy of the folder: the `scenarios/` folder (a five-pod "Triage Gauntlet", covered in §14) and a `02-kubectl-describe/demo-chart/` Helm chart (a `helm create` scaffold — Helm is next class, so it's only mentioned in §4). Both look like they were added to her repo after you copied it. They're covered here because they live in this session's folder.

---

0 ⚡ The 60-Second Version

1. **The question this whole class answers:** "*My Kubernetes application is not working. How do I find out why?*" — and the answer is **a process, not a guess**.
2. **Five commands, five questions.** `get` → *what is happening?* · `describe` → *what details explain it?* · `logs` → *what is the app saying?* · `exec` → *what can I see from inside?* · `events` → *what did Kubernetes try, and what happened?*
3. **The status column is a symptom, never the root cause.** `CrashLoopBackOff`, `ImagePullBackOff` and `Pending` each point you at a *different* next command.
4. `CrashLoopBackOff` = the image is fine, the container starts, **the app keeps dying**. Read `kubectl logs --previous`.
5. `ImagePullBackOff` = Kubernetes **can't even get the image** (wrong name, wrong tag, private registry). Read the **Events** in `describe`.

6. `Pending` = the scheduler **can't find a node** for the pod (no matching node, not enough CPU/RAM, taints, missing PVC). Read the `FailedScheduling` event.
7. **Service broken?** Follow the chain **pod labels** → **service selector** → **endpoints**. `ENDPOINTS <none>` means the selector matches nothing. The pod and the service can both be perfectly "healthy" and traffic still fails.
8. **DNS?** Test from **inside** a pod with `nslookup <service>`; the full name is `service.namespace.svc.cluster.local`. If DNS resolves but HTTP fails — **don't blame DNS**.

---

1 🗒️ Logistics — The Class That Wasn't Recorded

When	Sunday 21 September 2026, **4:15 PM**, 120 minutes
Header title	"Kubernetes 8" — the eighth Kubernetes session
What was captured	~30 seconds: "*can we start with the session… sharing my screen… are you able to see my screen?*"
What happened next	The recording stops. Nothing else exists.
What she screen-shared	Her `session-14-kubernetes-troubleshooting` folder — the same one you copied into this class folder

What's in your class folder:

File / folder	What it is
`README.md`, `01-…` to `09-…`, `mini-project/`	Your copy of her session folder
`01-kubectl-get/sample-workload.yaml`	Her `pod.yaml`, renamed (contents identical)
`09-service-dns-troubleshooting/service.yaml`	The **working** version (`app: web`) — her repo now has a deliberately broken one (§11)
`deployment.yaml`	**Your own** — a `yatri-backend` Python Deployment (reviewed in §15)
`service.yml`	**Your own** — an `nginx-service` NodePort (reviewed in §15)
`Readme.md-2`	Empty

HOMEWORK

The homework this session sets is the `mini-project/README.md` challenge: diagnose a broken pod and a broken service, fill in a five-question sheet, a troubleshooting table, and ten README questions. §13 walks through it and gives model answers in ✅ cards — try it yourself first.

---

2 🧭 The Troubleshooting Mindset

Her README opens with the one rule that matters: **when an application is not working, don't randomly run commands. Follow a process.**

PICTURE THIS

Picture a doctor, not a mechanic hitting things with a spanner.

Doctor	Kubernetes
Looks at you: pale, coughing	`kubectl get` — the status
Takes your temperature, blood pressure, history	`kubectl describe` — the details
Reads the nurse's chart: "*given X at 2 PM, reacted badly*"	**Events** — what the system tried
Asks you "*where does it hurt?*"	`kubectl logs` — what the app says
Listens with a stethoscope, takes a blood sample	`kubectl exec` — look from inside
Runs a lab test	`nslookup`, `curl`, `wget` — test connectivity
Diagnosis → prescription → follow-up visit	Root cause → **fix** → **verify**

A doctor who prescribes antibiotics before asking a single question is guessing. So is an engineer who runs `kubectl delete pod` the moment something goes red.

2.1 The 11-step process (from her README)

```
 1. Observe                    — something is wrong
 2. Identify the resource      — which pod / service / deployment?
 3. Check status               — kubectl get
 4. Check details              — kubectl describe
 5. Check events               — Events section / kubectl get events
 6. Check logs                 — kubectl logs (--previous)
 7. Enter container if possible — kubectl exec
 8. Test connectivity          — nslookup / curl / wget
 9. Find root cause
10. Fix
11. Verify                     — run step 3 again — did it actually work?
```

2.2 The five commands and the question each one answers

#	Command	The question	What it can't tell you
1	`kubectl get pods`	*"What is happening?"*	**Why**
2	`kubectl describe pod <name>`	*"What details can explain the problem?"*	What the app itself printed
3	`kubectl logs <name>`	*"What is the application saying?"*	Anything from *before* the container started (image pull, scheduling)
4	`kubectl exec -it <name> -- sh`	*"What can I see from inside the container?"*	Anything, if the container won't stay up
5	`kubectl get events`	*"What did Kubernetes try, and what happened?"*	Anything older than ~1 hour

LOCK THIS IN

Lock this in. The last column is why you need all five. Logs are useless for an image-pull failure (there's no container to log anything). Exec is useless for a crash loop (nothing stays up to enter). Each command covers the blind spot of the one before.

2.3 🗺️ The decision flow — which branch do I take?

Her README gives the "golden flow" as a straight line. In practice the **STATUS column** decides which branch you take first. This map combines her five per-problem flows (§8–§12) into one picture:

```
kubectl get pods  ──▶  read the STATUS column
│
├─ CrashLoopBackOff  (RESTARTS keeps climbing)                    §8
│     look:   kubectl logs --previous · describe → Last State, Exit Code
│     cause:  app error · bad command · missing env var · OOMKilled
│
├─ ErrImagePull / ImagePullBackOff  (RESTARTS stays 0)            §9
│     look:   kubectl describe pod → Events: "Failed to pull image"
│     cause:  wrong name · wrong tag · registry auth · network
│
├─ Pending  (NODE <none>)                                         §10
│     look:   kubectl describe pod → Events: FailedScheduling · get nodes
│     cause:  no matching node · not enough CPU/RAM · taint · unbound PVC
│
├─ Running 1/1 — but the Service is unreachable                   §11–12
│     look:   get pods --show-labels · describe svc · get endpoints
│             ENDPOINTS <none>?  yes → selector ≠ pod labels
│                                no  → nslookup <svc> from a pod
│                                        fails → wrong name / CoreDNS
│                                        works → targetPort · app · policy
│
└─ Running 1/1 — but the app misbehaves                          §6
      look:   kubectl logs -f · exec -- sh · curl localhost
      cause:  config · dependencies · env vars
                │
                ▼
      FIX  ──▶  kubectl get pods again  ──▶  VERIFY
```

GOOD TO KNOW

Filled in: the map itself is a synthesis — every box in it comes from her folder READMEs, but she presents them as separate flows. The right-most "Running but misbehaves" branch is from her `04-kubectl-exec` README ("*if localhost works, investigate Service, DNS, Network, Port*").

---

3 👀 `kubectl get` — "What Is Happening?"

Her one-line framing: "*Kubernetes, show me what is currently happening.*" It's the **first** command, every single time.

3.1 Reading the output

```
NAME       READY   STATUS    RESTARTS   AGE
get-demo   1/1     Running   0          10s
```

Column	Meaning	What should make you suspicious
NAME	Pod name	—
READY	ready containers / total containers	anything less than `n/n` (e.g. `0/1`)
STATUS	current state	anything that isn't `Running` or `Completed`
RESTARTS	how many times the container restarted	a number that **keeps going up**
AGE	how long the pod has existed	a pod that's always 30 seconds old is being recreated over and over

The three columns to read first: `STATUS`, `READY`, `RESTARTS`. Her example of a single glance telling you everything:

```
NAME       READY   STATUS             RESTARTS
my-app     0/1     CrashLoopBackOff   5
```

→ *Something is wrong with the container.* But `get` only says **what** — next you need `describe` and `logs` for **why**.

3.2 The variations

Command	Use it for
`kubectl get pods`	the quick view
`kubectl get pods -o wide`	+ **pod IP** and **node** — essential for service and scheduling problems
`kubectl get services` / `svc`	services
`kubectl get deployments` / `deploy`	deployments
`kubectl get nodes`	are the nodes `Ready`?
`kubectl get all`	pods + services + deployments + replicasets in one go
`kubectl get pods -w`	**watch** — keeps printing as things change
`kubectl get pods --show-labels`	the labels on each pod (critical in §11)

```
NAME       READY   STATUS    RESTARTS   AGE   IP           NODE
get-demo   1/1     Running   0          20s   10.244.0.5   minikube
```

(Your IP and node name will differ.)

GOOD TO KNOW

`kubectl get all` is not actually "all". It skips ConfigMaps, Secrets, Ingresses, PVCs and Endpoints, among others. Filled-in fact — but worth knowing before you trust it during an incident.

DO THIS YOURSELF

Lab 3.1 — `get` and watch a pod die live.

```
cd "Class-15-Kubernetes Troubleshooting/01-kubectl-get"
kubectl apply -f sample-workload.yaml     # her file is called pod.yaml — yours was renamed
# → pod/get-demo created

kubectl get pods                          # Running, 1/1, 0 restarts
kubectl get pods -o wide                  # note the IP and NODE
kubectl get all                           # what else is in the namespace?
kubectl get nodes                         # is minikube Ready?

# Terminal A
kubectl get pods -w
# Terminal B
kubectl delete pod get-demo
```

What to notice in terminal A: `Running` → `Terminating` → gone. That `-w` stream is how you watch a crash loop or a rollout happen in real time instead of re-typing `get` every two seconds.

LOCK THIS IN

`kubectl get` = **"What is happening?"** It gives the *current state*. It never gives the reason.

---

4 🔬 `kubectl describe` — "Why Is It Happening?"

If `get` says "*something is wrong*", `describe` asks "*what exactly happened?*"

```
get                          describe
 │                             │
 └── quick view                └── detailed investigation
```

4.1 The sections that matter

```
kubectl apply -f demo-pod.yaml
kubectl describe pod describe-demo
```

Section	What it tells you
Name / Namespace	Are you even looking at the right pod?
Labels	What a Service selector would need to match (§11)
Status	Pod phase: `Pending`, `Running`, `Succeeded`, `Failed`
IP	The pod's own IP — should show up in a Service's endpoints
Containers	Image, **State**, **Last State**, **Ready**, **Restart Count**, exit codes
Conditions	`PodScheduled`, `Initialized`, `ContainersReady`, `Ready` — True/False
Events	The timeline, at the bottom — the most important part

The **Containers** block on a healthy pod:

```
Containers:
  nginx:
    Image: nginx:1.27
    State: Running
    Ready: True
```

The **Events** block on a healthy pod:

```
Events:
  Type     Reason    Message
  Normal   Pulling   Pulling image
  Normal   Pulled    Successfully pulled image
  Normal   Created   Created container
  Normal   Started   Started container
```

PICTURE THIS

`describe` is the pod's medical file; Events are the last page. Most people read the first page (Name, Labels) and stop. The diagnosis is almost always on the **last** page. Scroll down.

4.2 It works on everything

```
kubectl describe deployment <deployment-name>
kubectl describe service <service-name>
kubectl describe node <node-name>     # capacity, allocated resources, taints — gold for Pending pods
```

4.3 The troubleshooting habit

Her README's rule: if you see `STATUS: Pending`, **don't immediately delete the pod.** Run `kubectl describe pod <pod-name>` and read the **Events**. Deleting the pod deletes the evidence — and a Deployment would just recreate it with the same problem anyway.

WATCH OUT

Small mismatch in her README. `02-kubectl-describe/README.md` says `kubectl apply -f pod.yaml`, but the file in that folder is **`demo-pod.yaml`**. Use `kubectl apply -f demo-pod.yaml` — the pod it creates is called `describe-demo`.

DO THIS YOURSELF

Lab 4.1 — Read a healthy pod's medical file.

```
cd "Class-15-Kubernetes Troubleshooting/02-kubectl-describe"
kubectl apply -f demo-pod.yaml
kubectl get pod                       # quick view
kubectl describe pod describe-demo    # the long view
```

Find, in order: **1.** the `Labels` line (`app=describe-demo`) · **2.** the pod `IP` · **3.** under Containers, `State: Running` and `Ready: True` · **4.** under Conditions, all four `True` · **5.** the four Normal events at the bottom. Then run `kubectl describe node minikube` and find the **Allocated resources** table — you'll need it in §10.

NOTE

About `demo-chart/` in her copy of this folder. Her repo also has a `02-kubectl-describe/demo-chart/` — the default scaffold that `helm create demo-chart` generates (Deployment, Service, HPA, Ingress, HTTPRoute, ServiceAccount templates and a `values.yaml`). It isn't used anywhere in this session's READMEs and isn't in your copy. **Helm is next class** (Class 16).

LOCK THIS IN

`kubectl get` → *what is happening?* · `kubectl describe` → *why is it happening?*

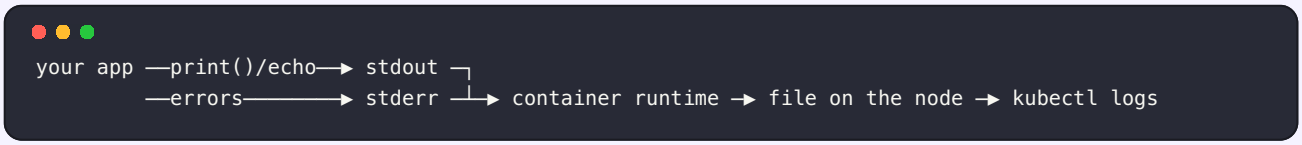
---

5 📜 `kubectl logs` — "What Is the Application Saying?"

Logs are the app talking. Her framing: "*Let me see what the application itself is saying.*"

PICTURE THIS

Where logs come from. Every process has two output pipes: **stdout** (normal output) and **stderr** (errors). Anything a container writes to those two pipes is captured by the container runtime on the node, and `kubectl logs` reads it back to you.



Consequence: **if your app writes its logs to a file inside the container** (say `/var/log/app.log`) instead of stdout, `kubectl logs` shows **nothing**. That's why container images send logs to stdout.

5.1 The demo pod

Her `03-kubectl-logs/pod.yaml` is a fake app — `busybox` printing a startup sequence and then a heartbeat every 5 seconds:

```
spec:
  containers:
    - name: app
      image: busybox:1.36
      command:
        - sh
        - -c
        - |
          echo "Application started"
          echo "Connecting to database..."
          echo "Database connection successful"
          echo "Application is running"
          while true; do
            echo "Application is healthy"
            sleep 5
          done
```

5.2 The four flavours

Command	What it does
`kubectl logs logs-demo`	everything printed so far
`kubectl logs -f logs-demo`	**follow** — stream new lines live (`Ctrl+C` to stop)
`kubectl logs logs-demo --previous`	logs of the **previous, dead** container — the crash-loop lifesaver
`kubectl logs logs-demo -c app`	pick one container in a multi-container pod

WHY IT WORKS

Why `--previous` exists. When a container crashes, Kubernetes starts a **brand-new** container in the same pod. Plain `kubectl logs` shows the *new* one — which may have printed nothing yet. The error message you need is in the *old* one's logs, and `--previous` is the only way back to it.

```
container #1: "Starting… ERROR: DB unreachable" ── dies
container #2: "Starting…"                          ── running now ← kubectl logs shows THIS
                                                    ← kubectl logs --previous shows #1
```

DO THIS YOURSELF

Lab 5.1 — Read, follow, and pick a container.

```
cd "Class-15-Kubernetes Troubleshooting/03-kubectl-logs"
kubectl apply -f pod.yaml
kubectl get pod logs-demo          # 1/1 Running
kubectl logs logs-demo
```

Expected:

```
Application started
Connecting to database...
Database connection successful
Application is running
Application is healthy
Application is healthy
```

Then: **1.** `kubectl logs -f logs-demo` — watch a new `healthy` line every 5 seconds, `Ctrl+C` to stop. **2.** `kubectl logs logs-demo -c app` — same output, because the only container is called `app`. **3.** `kubectl logs logs-demo -c wrong` — read the error: it lists the valid container names. **4.** `kubectl logs logs-demo --tail=2` *(filled in)* — just the last two lines.

LOCK THIS IN

`kubectl logs` = **"What is the application saying?"** Often the first place to look when an app crashes or misbehaves.

---

6 🚪 `kubectl exec` — "Let Me Check From Inside"

`exec` runs a command **inside** a running container. Her framing: "*Let me enter the container and check what is happening from inside.*"

PICTURE THIS

`exec` is `docker exec` for pods. Same idea you used in the Docker classes — but you don't need to know which node the pod is on; the API server routes you there.

6.1 The two shapes

```
# 1. Open an interactive shell
kubectl exec -it exec-demo -- bash
# root@exec-demo:/#

# 2. Run one command, get the output, done
kubectl exec exec-demo -- hostname
kubectl exec exec-demo -- ls /usr/share/nginx/html
kubectl exec exec-demo -- cat /etc/hosts
```

WHY IT WORKS

What the `--` is for. Everything **before** `--` is for `kubectl`; everything **after** it is the command to run inside the container. Without it, `kubectl` would try to interpret `ls -la` as *its own* `-l` and `-a` flags.

```
kubectl exec -it exec-demo  --  ls -la /usr/share/nginx/html
└──── kubectl's part ─────┘     └──── runs inside the pod ────┘
```

And `-it` = `-i` (keep stdin open) + `-t` (give me a terminal). You need both for a shell; you need neither for a one-shot command.

6.2 Why it's useful — narrowing the problem down

Her scenario: **the Service is not working.** Exec into the pod and run `curl localhost`.

```
curl localhost inside the pod works?
        |
   yes ─┴─ no
    |       |
    |       └─▶ the APP is broken — logs, config, nginx -T
    ▼
the app is fine — so investigate what's BETWEEN you and it:
Service · DNS · Network · Port
```

That one test cuts the search space in half.

Other inside-checks from her README:

Inside the container	Proves
`ls /usr/share/nginx/html`	the files you expected were actually shipped
`curl localhost`	the app answers on its own port
`nginx -T`	dumps nginx's *full* effective config
`cat /etc/hosts`	the pod's own name → IP mapping

WATCH OUT

`exec` **needs a container that's running.** If the container is crash-looping, there's nothing stable to enter — you'll keep getting kicked out or get `container not found`. For those cases her README points you to `kubectl logs`, `kubectl describe` and `kubectl debug`.

GOOD TO KNOW

Filled in — what `kubectl debug` does. It attaches a temporary **extra** container (with your choice of image — e.g. one full of tools) to a pod, or makes a copy of the pod you can poke at. Useful when the real image has no shell at all (distroless) or won't stay up. Example: `kubectl debug -it exec-demo --image=busybox:1.36 --target=nginx`.

GOOD TO KNOW

`bash` **vs** `sh`**.** `nginx:1.27` is Debian-based and has `bash`. Tiny images like `busybox` and most `alpine` images only have `sh`. If `-- bash` fails with "*executable file not found*", try `-- sh`. That's why her generic checklist says `-- sh`.

DO THIS YOURSELF

Lab 6.1 — Look around inside nginx.

```
cd "Class-15-Kubernetes Troubleshooting/04-kubectl-exec"
kubectl apply -f pod.yaml
kubectl get pod exec-demo                  # 1/1 Running

kubectl exec -it exec-demo -- bash
  ls
  ls /usr/share/nginx/html                 # 50x.html  index.html
  curl localhost                           # the "Welcome to nginx!" HTML
  nginx -T | head -30                      # nginx's real config
  exit

kubectl exec exec-demo -- hostname         # → exec-demo
kubectl exec exec-demo -- cat /etc/hosts
```

Bonus — break it from inside: `kubectl exec exec-demo -- rm /usr/share/nginx/html/index.html`, then `kubectl exec exec-demo -- curl -s localhost` — you now get a `403 Forbidden`. The pod is still `1/1 Running`. **"Running" doesn't mean "working".** Fix: `kubectl delete pod exec-demo && kubectl apply -f pod.yaml` — a fresh container gets a fresh filesystem.

LOCK THIS IN

`kubectl exec` = **"Let me check from INSIDE the container."**

---

7 Events — Kubernetes' Activity Log

Her one-liner: **Events are "Kubernetes' activity log for a resource."** Every time the scheduler places a pod, the kubelet pulls an image, a container starts or a probe fails, an Event is written.

```
kubectl apply -f pod.yaml          # pod/events-demo created
kubectl get events
```

```
TYPE     REASON     OBJECT              MESSAGE
Normal   Scheduled  Pod/events-demo     Successfully assigned...
Normal   Pulling    Pod/events-demo     Pulling image...
Normal   Pulled     Pod/events-demo     Successfully pulled...
Normal   Created    Pod/events-demo     Created container...
Normal   Started    Pod/events-demo     Started container...
```

WATCH IT HAPPEN

Watch a healthy pod being born — through its Events.

```
Frame 1  Scheduled  kube-scheduler: "events-demo → node minikube"
Frame 2  Pulling    kubelet: "fetching nginx:1.27 from the registry…"
Frame 3  Pulled     kubelet: "got it" (or "already present on machine")
Frame 4  Created    runtime: "container created"
Frame 5  Started    runtime: "container running"   → STATUS: Running
```

Every failure in this class is just **one of these frames going wrong**: Frame 1 fails → `Pending` (§10). Frame 2 fails → `ImagePullBackOff` (§9). Frame 5 succeeds but the app dies → `CrashLoopBackOff` (§8).

7.1 All the ways to read them

Command	Use
`kubectl get events`	every event in the namespace
`kubectl get events --sort-by=.lastTimestamp`	in time order — much easier to follow
`kubectl get events --field-selector type=Warning`	**only the problems** — cut the noise during an incident
`kubectl describe pod <name>`	the same events, filtered to that pod, at the bottom
`kubectl events`	the newer dedicated command
`kubectl events --watch`	stream events live
`kubectl events --for pod/events-demo`	events for one specific object

7.2 The four facts from her "Key Learning" list

- Events are the **fastest way to understand what the control plane is doing**.
- Filter with `--field-selector type=Warning` during incidents.
- **Events expire from `etcd` after 1 hour by default.** If a pod broke yesterday, the events are gone — logs and your own notes are what's left.
- The message on a `FailedScheduling` event tells you **exactly** why a pod is stuck in `Pending`.

PICTURE THIS

`Normal` vs `Warning`. Events only come in two types. `Normal` = "*I did the thing.*" `Warning` = "*I tried the thing and it didn't work.*" During an incident you almost never care about the `Normal` ones.

DO THIS YOURSELF

Lab 7.1 — Events, sorted, filtered, live.

```
cd "Class-15-Kubernetes Troubleshooting/05-events"
# Terminal A
kubectl events --watch
# Terminal B
kubectl apply -f pod.yaml                          # watch Scheduled → Pulling → Pulled → Created → Started
kubectl get events --sort-by=.lastTimestamp
kubectl events --for pod/events-demo
kubectl describe pod events-demo | tail -15        # same events, pod-scoped

# Now create a Warning on purpose and filter for it:
kubectl run oops --image=nginx:no-such-tag
kubectl get events --field-selector type=Warning
kubectl delete pod oops
```

LOCK THIS IN

`get` → current status · `describe` → detailed information · **Events → what Kubernetes tried to do and what happened.**

---

8 🔁 `CrashLoopBackOff` — The App Keeps Dying

Her folder's three-word plan for every broken-pod lab: **See problem → Find reason → Fix problem.**

8.1 What it means

The container is repeatedly starting and failing. The image was pulled fine, the container started fine — and then the *app inside it* exited. Kubernetes restarts it. It exits again. Kubernetes waits a bit longer, restarts it again.

WATCH IT HAPPEN

Watch a crash loop happen, frame by frame.

```
Frame 1  Container starts            STATUS: Running      RESTARTS: 0
Frame 2  App prints, then exit 1     STATUS: Error        RESTARTS: 0
Frame 3  Kubelet restarts it         STATUS: Running      RESTARTS: 1
Frame 4  Crashes again               STATUS: Error
Frame 5  Kubelet: "I'll wait 10s"    STATUS: CrashLoopBackOff
Frame 6  Restart → crash             RESTARTS: 2   → wait 20s
Frame 7  Restart → crash             RESTARTS: 3   → wait 40s
…        … the wait keeps doubling …                  → capped at 5 min
```

"BackOff" is that growing wait. `CrashLoopBackOff` literally means "*it crashed in a loop, so I'm backing off before trying again.*"

GOOD TO KNOW

Filled in — the back-off numbers. Kubernetes' documented restart back-off starts at **10 s** and doubles (10, 20, 40, 80, 160 s) up to a **5-minute cap**. If a container then runs for **10 minutes** without crashing, the timer resets. Her README just calls it "*part of Kubernetes' backoff behavior.*"

8.2 The broken pod

```yaml
# 06-crashloopbackoff/broken-pod.yaml
spec:
  containers:
    - name: app
      image: busybox:1.36
      command:
        - sh
        - -c
        - |
          echo "Application starting..."
          echo "Something went wrong!"
          exit 1
```

```
kubectl apply -f broken-pod.yaml
kubectl get pod crash-demo
```

```
NAME         READY   STATUS             RESTARTS
crash-demo   0/1     CrashLoopBackOff   3
```

8.3 The investigation, in her order

Step	Command	What to look for
1	`kubectl describe pod crash-demo`	**State**, **Last State** (`Terminated` , `Reason: Error` , `Exit Code: 1`), **Restart Count**, **Events** (`Back-off restarting failed container`)
2	`kubectl logs crash-demo`	`Application starting...` / `Something went wrong!`
3	`kubectl logs crash-demo --previous`	the same lines from the **dead** container — useful when the current one has already restarted
4	Read the YAML	`exit 1`

Root cause: the command ends in `exit 1`. Exit code 1 means "*the command failed*" — we deliberately told the app to crash.

PICTURE THIS

Filled in — exit codes, the 30-second version. When a process ends it hands back a number. `describe` shows it as **Exit Code** under *Last State*.

Exit code	Means	Typical cause
0	success	the process finished its job normally
1	generic error	app error, uncaught exception, `exit 1`
126	found but can't execute	missing `+x` permission
127	command not found	typo in `command:`, binary not in image
137	killed by SIGKILL (128 + 9)	**OOMKilled** — out of memory (§14, scenario 5)
143	killed by SIGTERM (128 + 15)	normal shutdown request

Anything above 128 = "*killed by signal (code − 128)*". That one rule decodes most of them.

8.4 The fix

```
# 06-crashloopbackoff/fixed-pod.yaml  — only the script changes
        - |
          echo "Application starting..."
          echo "Application is healthy"
          sleep 3600
```

```
kubectl delete pod crash-demo
kubectl apply -f fixed-pod.yaml
kubectl get pod crash-demo          # 1/1 Running
kubectl logs crash-demo             # Application starting... / Application is healthy
```

WHY IT WORKS

Why delete first instead of just `apply`-ing the fixed file? A bare Pod's spec is almost entirely **immutable** — you can't change its `command` after it's created. `kubectl apply -f fixed-pod.yaml` on top of the running broken pod fails with "*Forbidden: pod updates may not change fields other than…*". Delete-then-apply sidesteps that. (With a **Deployment** you'd just `apply` — it rolls out new pods for you.)

CLARIFIED

Clarified, since "fixed" depends on the app never exiting. Her fixed pod works because `sleep 3600` keeps the process alive. But **pods default to `restartPolicy: Always`** — so even a *successful* exit (code 0) gets restarted. After 3600 s the `sleep` ends, the container exits 0, and `RESTARTS` ticks to 1. If a container exits 0 quickly and repeatedly, you'll see `Completed` and then… **`CrashLoopBackOff`**, with no error anywhere. So `CrashLoopBackOff` really means "*the container keeps stopping*", not strictly "*keeps failing*". For her course, give her definition ("*repeatedly starting and failing*"); in real life, also check whether the process is simply **finishing**. A one-off task should be a **Job**, not a Pod/Deployment.

8.5 The flow and the real causes

```
kubectl get pod → STATUS? → CrashLoopBackOff
      → kubectl describe pod → kubectl logs → kubectl logs --previous
      → find root cause → fix
```

`CrashLoopBackOff` **is a symptom, not the root cause.** Her list of what it can actually be:

Cause	Where you'd see it
Application error	logs — stack trace
Wrong command	Exit Code 127 / `executable file not found` in describe
Missing environment variable	logs — e.g. `DATABASE_URL is MISSING` (§14, scenario 1)
Configuration problem	logs — bad config file, wrong port
Failed dependency	logs — `connection refused` to the DB
Bad probe	Events — `Liveness probe failed` → kubelet kills it
Permission problem	logs — `permission denied`

WATCH OUT

Always investigate instead of simply restarting the pod. Deleting a crash-looping pod just gets you a new one that crashes the same way — with the old logs gone.

DO THIS YOURSELF

Lab 8.1 — Break it, diagnose it, fix it.

```
cd "Class-15-Kubernetes Troubleshooting/06-crashloopbackoff"
kubectl apply -f broken-pod.yaml
kubectl get pod crash-demo -w        # watch RESTARTS climb and the gaps grow; Ctrl+C after ~2 min

kubectl describe pod crash-demo      # find: Last State Terminated, Exit Code 1, Back-off event
kubectl logs crash-demo
kubectl logs crash-demo --previous

kubectl apply -f fixed-pod.yaml      # ← try this first: read the "Forbidden" error
kubectl delete pod crash-demo
kubectl apply -f fixed-pod.yaml
kubectl get pod crash-demo           # 1/1 Running
kubectl logs crash-demo              # Application is healthy
```

Extension — break it a different way. Copy `broken-pod.yaml`, change `exit 1` to `exit 0`. Apply it. Watch: `Completed` → `CrashLoopBackOff`, with **no error message anywhere**. That's the 🔍 note above, live.

9 📦 `ImagePullBackOff` — Kubernetes Can't Get the Image

9.1 What it means

Kubernetes tried to **download** the image and couldn't. The container never even started — so there are **no logs**.

WATCH IT HAPPEN

Watch an image pull fail, frame by frame.

```
Frame 1  Scheduled           pod placed on minikube              STATUS: Pending
Frame 2  Pulling             "nginx:this-image-does-not-exist"…   STATUS: ContainerCreating
Frame 3  Failed              registry: "manifest unknown"         STATUS: ErrImagePull
Frame 4  BackOff             kubelet waits, then retries          STATUS: ImagePullBackOff
Frame 5  Pulling → Failed    same answer                          STATUS: ErrImagePull
Frame 6  BackOff (longer)                                         STATUS: ImagePullBackOff
…        flips between the two forever, waits growing like §8     RESTARTS: 0 — always
```

`ErrImagePull` **= the attempt just failed.** `ImagePullBackOff` **= waiting before the next attempt.** Same problem, two faces.

9.2 The broken pod

```yaml
# 07-imagepullbackoff/broken-pod.yaml
spec:
  containers:
    - name: app
      image: nginx:this-image-does-not-exist
```

```
kubectl apply -f broken-pod.yaml
kubectl get pod image-demo
```

```
NAME         READY   STATUS             RESTARTS
image-demo   0/1     ErrImagePull       0
# …after some retries:
image-demo   0/1     ImagePullBackOff   0
```

9.3 Diagnose — describe, then Events

```
kubectl describe pod image-demo
```

The **Events** section shows something like `Failed to pull image "nginx:this-image-does-not-exist" … not found` *(exact wording depends on runtime and cluster)*. Then check the YAML: **that tag doesn't exist.** Root cause found.

PICTURE THIS

Why `logs` is useless here. Logs are what a *running app* prints. Here there's no app — there's not even a container yet. Only the **kubelet** knows what went wrong, and it reports through **Events**. That's the single most useful rule for this error: **image problem → describe → Events.**

9.4 The fix

```
# 07-imagepullbackoff/fixed-pod.yaml
      image: nginx:1.27
```

```
kubectl delete pod image-demo
kubectl apply -f fixed-pod.yaml
kubectl get pod image-demo           # 1/1 Running
```

GOOD TO KNOW

Filled in — the image field is the exception. Unlike `command`, a pod's container `image` **is** mutable, so `kubectl apply -f fixed-pod.yaml` without deleting would actually work here. Her delete-then-apply habit is still the safe default for bare pods.

9.5 Other causes (her list) and how to spot each

Cause	Typical Events message
Wrong image **name**	`pull access denied … repository does not exist or may require authorization`
Wrong image **tag**	`manifest … not found` / `manifest unknown`
Private registry auth problem	`unauthorized` / `authentication required` → needs an `imagePullSecret`
Registry unavailable	timeouts, `connection refused`
Network problem	`i/o timeout`, DNS failure resolving the registry
Image doesn't exist	as wrong name/tag

(Message wordings are filled in as typical examples — exact text varies by runtime.)

```
ImagePullBackOff → describe pod → Events → check image NAME → check TAG → check REGISTRY ACCESS → fix
```

9.6 The two BackOffs side by side

	`CrashLoopBackOff`	`ImagePullBackOff`
Her one-liner	**Container starts but keeps failing**	**Kubernetes cannot get the image**
Got as far as	Frame 5 — started	Frame 2 — pulling
`RESTARTS`	climbs	stays **0**
Logs?	✅ yes — `logs` / `logs --previous`	❌ none — no container ever ran
Look at	logs first, then describe	**describe → Events**
Usually whose fault?	the app / its config	the YAML's `image:` line or registry access

DO THIS YOURSELF

Lab 9.1 — Three ways to fail a pull.

```
cd "Class-15-Kubernetes Troubleshooting/07-imagepullbackoff"
kubectl apply -f broken-pod.yaml
kubectl get pod image-demo -w             # ErrImagePull ⇄ ImagePullBackOff; Ctrl+C
kubectl logs image-demo                   # read the error — "waiting to start"
kubectl describe pod image-demo           # Events → Failed to pull image
kubectl delete pod image-demo
kubectl apply -f fixed-pod.yaml && kubectl get pod image-demo
```

Now compare the Events wording for three different mistakes:

```
kubectl run bad-tag  --image=nginx:9.9.9-nope          # wrong tag
kubectl run bad-name --image=ngnix:1.27                # typo in the name
kubectl run bad-reg  --image=no.such.registry.io/app:1  # registry doesn't exist
kubectl describe pod bad-tag  | tail -5
kubectl describe pod bad-name | tail -5
kubectl describe pod bad-reg  | tail -5
kubectl delete pod bad-tag bad-name bad-reg
```

Write down which message goes with which mistake. Next time you see it in production, you'll know in one glance.

---

10 ⏳ `Pending` Pods — No Node Will Take It

10.1 What it means

```
Pod created → Scheduler tries to find a node → Cannot schedule → Pending
```

Pending = the pod has not successfully been scheduled/started yet. It's sitting in a queue because no node passes the scheduler's checks.

PICTURE THIS

The scheduler is a hotel receptionist. A guest (pod) arrives with requirements: "*a room on floor 7*" (nodeSelector), "*two beds*" (CPU/RAM requests), "*I'm allergic to cats*" (taints). The receptionist checks every room (node). If none fits, the guest waits in the lobby — **Pending**. The receptionist leaves a note at the desk saying why: that's the `FailedScheduling` event.

10.2 The broken pod

```
# 08-pending-pods/broken-pod.yaml
spec:
  nodeSelector:
    kubernetes.io/hostname: node-that-does-not-exist
  containers:
    - name: nginx
      image: nginx:1.27
```

```
kubectl apply -f broken-pod.yaml
kubectl get pod pending-demo
```

```
NAME           READY   STATUS    RESTARTS   AGE
pending-demo   0/1     Pending   0          10s
```

(It stays Pending forever.)

10.3 Diagnose

```
kubectl describe pod pending-demo
kubectl get nodes
```

```
NAME       STATUS   ROLES
minikube   Ready    control-plane
```

The pod demands a node whose hostname label is `node-that-does-not-exist`. The only node is `minikube`. **No match → Pending.**

WATCH IT HAPPEN

The scheduler's decision, frame by frame.

```
Frame 1  Pod pending-demo arrives, NODE: <none>
Frame 2  Scheduler lists nodes:                    [minikube]
Frame 3  Filter: nodeSelector hostname=node-that-does-not-exist?
         minikube has hostname=minikube            ✗
Frame 4  0 nodes left
Frame 5  Event: Warning FailedScheduling
         "0/1 nodes are available: 1 node(s) didn't match Pod's node affinity/selector."
Frame 6  Condition PodScheduled: False → retry later … forever
```

(The FailedScheduling wording is the standard scheduler message, filled in.)

Her checklist for spotting it in `describe`:

- `Node: <none>` — the pod is waiting on `kube-scheduler`.
- Conditions: `PodScheduled: False`.
- Events: the message under `Warning FailedScheduling` — it tells you *exactly* why.
- Most frequent real causes: **CPU/RAM request over-allocation** and **invalid node selectors**.

10.4 The fix

The fixed pod simply drops the `nodeSelector`:

```
kubectl delete pod pending-demo
kubectl apply -f fixed-pod.yaml
kubectl get pod pending-demo        # 1/1 Running
```

10.5 Other reasons a pod stays Pending (her list, with the jargon unpacked)

Cause	What it is	FailedScheduling says roughly
Insufficient CPU	the pod's `requests.cpu` is more than any node has free	`Insufficient cpu`
Insufficient memory	same, for `requests.memory`	`Insufficient memory`
Node selector mismatch	`nodeSelector` label matches no node	`didn't match Pod's node affinity/selector`
Affinity rules	a fancier, more flexible nodeSelector ("*prefer*"/"*require*" *nodes with label X*)	same as above
Taints & tolerations	a **taint** is a "keep out" sign on a node; only pods with a matching **toleration** may land there	`had untolerated taint`
PVC not available	the pod needs a PersistentVolumeClaim that isn't bound yet	`unbound immediate PersistentVolumeClaims`
Scheduling constraints	anything else that filters nodes out	—

(Message fragments filled in as typical wording.)

PICTURE THIS

Requests vs limits, in one line each (needed for the CPU/RAM causes and for §14). `requests` = what the pod *reserves* — the scheduler uses it to decide where it fits. `limits` = the ceiling it may *use* — exceed the memory limit and it's killed (OOMKilled). `cpu: "100m"` = one-tenth of a core; `memory: "128Mi"` = 128 mebibytes.

CLARIFIED

Clarified, since "Pending" appears in two places. The `STATUS` column in `kubectl get` and the pod **phase** in `describe` aren't the same thing. A pod stuck in `ImagePullBackOff` or `ContainerCreating` is *also* in phase **Pending** — it's scheduled, but its containers haven't started. Her definition covers this ("*not successfully been scheduled/started yet*"). The practical tell: `STATUS: Pending` + `Node: <none>` = a scheduling problem (this section). **Phase Pending + a node assigned** = an image or volume problem (§9).

```
Pending → describe pod → check Events → check nodes → check resources
        → check selectors / affinity → check PVC if used → fix
```

DO THIS YOURSELF

Lab 10.1 — Pending two ways.

```
cd "Class-15-Kubernetes Troubleshooting/08-pending-pods"
kubectl apply -f broken-pod.yaml
kubectl get pod pending-demo -o wide         # NODE: <none>
kubectl describe pod pending-demo            # Conditions + FailedScheduling event
kubectl get nodes --show-labels              # see the real kubernetes.io/hostname value
kubectl delete pod pending-demo
kubectl apply -f fixed-pod.yaml && kubectl get pod pending-demo
```

Round 2 — fix it the other way. Instead of deleting the selector, point it at the real node: edit a copy of `broken-pod.yaml`, change `node-that-does-not-exist` to `minikube`, apply. It schedules. Same symptom, two valid fixes — the right one depends on whether the selector was a typo or a mistake.

Round 3 — run out of resources. Run `kubectl describe node minikube` and read *Allocatable*. Then ask for more than that:

```
kubectl run greedy --image=nginx:1.27 --overrides='{"spec":{"containers":
[{"name":"greedy","image":"nginx:1.27","resources":{"requests":{"cpu":"64"}}}]}}'
kubectl describe pod greedy | tail -4        # Insufficient cpu
kubectl delete pod greedy
```

11 🔌 Service Troubleshooting — Labels → Selector → Endpoints

This is the one her README calls **"a very common problem"**: the pods are healthy, the Service exists, and traffic still fails.

PICTURE THIS

A Service is a WhatsApp group defined by a rule, not a member list. The Service never lists pods by name. It says "*everyone with the tag* `app=web` *is in this group.*" Kubernetes keeps re-checking which pods carry that tag, and the current member list is the **Endpoints**. Misspell the tag in the rule → the group is **empty** → messages go nowhere. Nobody is sick; the group just has no members.

```
Pod labels ──must match──▶ Service selector ──produces──▶ Endpoints (pod IP:port list)
 app=web                    app=web                        10.244.0.5:80, 10.244.0.6:80
```

11.1 Build the working version

```yaml
# 09-service-dns-troubleshooting/deployment.yaml (key parts)
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web          # ← the label
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

```yaml
# 09-service-dns-troubleshooting/service.yaml — your copy
kind: Service
metadata:
  name: web-service
spec:
  selector:
    app: web              # ← must equal the label
  ports:
    - port: 80
      targetPort: 80      # ← must equal the containerPort
  type: ClusterIP
```

```
kubectl apply -f deployment.yaml
kubectl get pods                    # web-xxxxxxxxxx-xxxxx ×2, Running
kubectl apply -f service.yaml
kubectl get service                 # web-service   ClusterIP   10.x.x.x
kubectl describe service web-service   # check Selector, Port, TargetPort, Endpoints
kubectl get endpoints web-service
```

```
NAME          ENDPOINTS
web-service   10.244.0.5:80,10.244.0.6:80
```

Those two IPs are exactly the two pods' IPs from `kubectl get pods -o wide`. That's the proof the chain is intact.

WATCH OUT

Her repo's `service.yaml` and your copy differ — on purpose. Your copy has `selector: app: web` (working). Her repo's current version has `app: web-ahsgdf` — a nonsense label that matches nothing. It looks like she edited the file during the class to break it live (the transcript can't confirm this). If you clone her repo fresh and run this lab, `kubectl get endpoints web-service` will show `<none>` straight away — which is now your cue to apply everything below.

11.2 Break it on purpose

```yaml
# 09-service-dns-troubleshooting/broken-service.yaml
metadata:
  name: broken-service
spec:
  selector:
    app: does-not-exist
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

```
kubectl apply -f broken-service.yaml
kubectl get service broken-service       # exists, has a ClusterIP — looks fine!
kubectl get endpoints broken-service
```

```
NAME             ENDPOINTS
broken-service   <none>
```

Why? Service says `app: does-not-exist`; the pods say `app: web`. No pod matches → no endpoints → nowhere to send traffic.

Her generalisation — pod labelled `app: backend`, Service selecting `app: frontend`:

```
Service ── selector does not match ──▶ No endpoints ──▶ No application traffic
```

LOCK THIS IN

The pod can be perfectly healthy. The Service can be perfectly created. Traffic still fails. `ENDPOINTS <none>` is the single most important thing to check when a Service "doesn't work".

11.3 Her 7-step Service checklist

#	Check	Command
1	Pods running?	`kubectl get pods`
2	What labels do they have?	`kubectl get pods --show-labels`
3	What does the Service select?	`kubectl describe service <name>` → `Selector:`
4	Did it find any pods?	`kubectl get endpoints <name>`
5	Does the name resolve?	`nslookup <service-name>` (from a pod)
6	Does HTTP work?	`wget` / `curl` (from a pod)
7	Is CoreDNS alive?	`kubectl get pods -n kube-system`

```
NAME                   LABELS
web-xxxxxxxxxx-xxxxx   app=web
web-xxxxxxxxxx-yyyyy   app=web
```

…compared with `Selector: app=web` in `describe service`. **They must match.**

11.4 The fix

```
kubectl delete service broken-service
kubectl get service web-service
kubectl get endpoints web-service     # Pod IPs are back
```

GOOD TO KNOW

Filled in — `Endpoints` vs `EndpointSlices`. On newer clusters (Kubernetes 1.33+), `kubectl get endpoints` prints a deprecation warning: the old `Endpoints` API is being replaced by **EndpointSlices**. It still works and shows the same answer. The modern equivalent: `kubectl get endpointslices -l kubernetes.io/service-name=web-service`.

INTERVIEW ALERT

Interview alert — the four places a Service can silently break.

Break	Symptom	Check
selector ≠ pod labels	`ENDPOINTS <none>`	`--show-labels` vs `describe svc`
`targetPort` ≠ `containerPort`	endpoints present, but `connection refused`	`describe svc` TargetPort vs pod's port
pods not **Ready**	endpoints empty (unready pods are excluded)	`kubectl get pods` READY column
wrong Service **name**/namespace in the client	DNS fails	`nslookup` (§12)

12 🌐 DNS Troubleshooting — Can the Pods Find Each Other by Name?

PICTURE THIS

Cluster DNS is the office phonebook. Pods don't remember each other's IPs (pod IPs change every restart). They ask the phonebook: "*what's the number for* `web-service`*?*" The phonebook is **CoreDNS**, a pair of pods in the `kube-system` namespace. Every new Service is added automatically.

12.1 The DNS name structure

```
web-service . default . svc . cluster.local
└─ service ┘ └ namespace ┘ └fixed┘ └ cluster domain ┘
```

You type (from a pod in `default`)	Works?	Why
`web-service`	✅	same namespace — the search list fills in the rest
`web-service.default`	✅	namespace given
`web-service.default.svc.cluster.local`	✅	the full **FQDN** — works from **any** namespace
`web-service` from a pod in **another** namespace	❌	it searches *its own* namespace first — use `web-service.default`

12.2 The DNS test pod

```yaml
# 09-service-dns-troubleshooting/dns-test-pod.yaml
metadata:
  name: dns-test
spec:
  containers:
    - name: dns-test
      image: registry.k8s.io/e2e-test-images/dnsutils:1.3
      command:
        - sleep
        - "3600"
```

A pod whose only job is to **sit there with DNS tools in it** (`nslookup`, `dig`) so you can `exec` tests from *inside* the cluster. That `registry.k8s.io/e2e-test-images/dnsutils` image is the one the official Kubernetes DNS-debugging guide uses.

12.3 The tests, in order

```bash
kubectl apply -f dns-test-pod.yaml
kubectl get pod dns-test                                    # 1/1 Running

# 1. Does the short name resolve?
kubectl exec -it dns-test -- nslookup web-service
# 2. Does the FQDN resolve?
kubectl exec -it dns-test -- nslookup web-service.default.svc.cluster.local
# 3. Does HTTP actually reach a pod?
kubectl exec dns-test -- wget -qO- http://web-service       # nginx HTML
# 4. What DNS server is this pod using?
kubectl exec -it dns-test -- cat /etc/resolv.conf
# 5. Is CoreDNS running?
kubectl get pods -n kube-system                             # coredns-xxxxx 1/1 Running
# 6. What is CoreDNS complaining about?
kubectl logs -n kube-system -l k8s-app=kube-dns
```

GOOD TO KNOW

Filled in — if `wget` isn't in that image. The `dnsutils` image is built for DNS tools; if `wget` comes back as "*executable file not found*", use a throwaway busybox pod for step 3 instead: `kubectl run tmp --rm -it --image=busybox:1.36 --restart=Never -- wget -qO- http://web-service`. The `--rm` deletes it when you exit.

When step 3 works, it proves the whole chain:

```
DNS works → Service resolves → Service routes traffic → Pod responds
```

WHY IT WORKS

What `/etc/resolv.conf` shows and why it matters. Typical contents *(filled in — values vary)*:

```
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
```

`nameserver` is the ClusterIP of the `kube-dns` Service (CoreDNS). The `search` line is why the short name `web-service` works: the resolver silently tries `web-service.default.svc.cluster.local` first. If `nameserver` points somewhere odd, DNS is broken **for the whole pod**.

WHY IT WORKS

Why `-l k8s-app=kube-dns`? CoreDNS replaced the older `kube-dns` years ago, but kept the old **label** for compatibility. So you find CoreDNS pods by the label `k8s-app=kube-dns`. Same label trick as §11, used on system pods.

12.4 The important difference — don't blame DNS

Her rule: if this **works**:

```
kubectl exec dns-test -- nslookup web-service
```

but this **fails**:

```
kubectl exec dns-test -- wget -qO- http://web-service
```

…then **DNS is fine**. The name resolved. The problem is further along:

Suspect	How to confirm
Service selector	`kubectl get endpoints` → `<none>`
Endpoints	same
`targetPort`	`describe svc` vs the pod's `containerPort`
Application	`exec` into the pod → `curl localhost` (§6)
Network policy	`kubectl get networkpolicy` — something may be blocking traffic

LOCK THIS IN

Do not immediately blame DNS. Resolve first; if it resolves, DNS did its job.

Her Key Learning chain:

```
Pod — label ⟶ Service selector ⟶ Endpoints ⟶ Service IP ⟶ DNS name
```

Read it right to left when debugging: name resolves? → to the right IP? → that IP has endpoints? → endpoints match labels?

DO THIS YOURSELF

Lab 12.1 — The full Service + DNS drill, including both failures.

```
cd "Class-15-Kubernetes Troubleshooting/09-service-dns-troubleshooting"
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl apply -f dns-test-pod.yaml
kubectl get pods -o wide --show-labels              # note the two web-… IPs
kubectl get endpoints web-service                   # those same two IPs

kubectl exec -it dns-test -- nslookup web-service   # ✅ resolves
kubectl exec dns-test -- wget -qO- http://web-service   # ✅ nginx HTML
```

Failure A — the selector (DNS works, traffic doesn't):

```
kubectl apply -f broken-service.yaml
kubectl get endpoints broken-service                    # <none>
kubectl exec -it dns-test -- nslookup broken-service    # ✅ STILL resolves!
kubectl exec dns-test -- wget -qO- -T 3 http://broken-service   # ❌ fails
kubectl get pods --show-labels                          # app=web
kubectl describe service broken-service | grep Selector  # app=does-not-exist
kubectl delete service broken-service
```

That's her "don't blame DNS" rule proved: the name resolved fine, there were just no pods behind it.

Failure B — the name (DNS genuinely fails):

```
kubectl exec -it dns-test -- nslookup web-servce          # typo → NXDOMAIN / can't find
kubectl exec -it dns-test -- nslookup web-service.prod    # wrong namespace → can't find
```

Failure C — the port (endpoints exist, still fails): copy `service.yaml`, change `targetPort: 80` → `8080`, apply. `get endpoints` now shows `…:8080`, and `wget` gets `connection refused`. Put it back to 80.

Finally, check the phonebook itself:

```
kubectl exec -it dns-test -- cat /etc/resolv.conf
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns
```

13 🧩 The Mini-Project — Troubleshooting Challenge

This is the session's homework (`mini-project/README.md`). The scenario: **a simple nginx app — a Deployment, a Service, its Pods — and your team reports that something is wrong.** Your loop:

```
Deploy → Observe → Break → Investigate → Find root cause → Fix → Verify
```

13.1 The files

File	Contents
`deployment.yaml`	`troubleshooting-app`, 2 replicas, label `app: troubleshooting-app`, `nginx:1.27` on port 80
`service.yaml`	`troubleshooting-service`, ClusterIP, selector `app: troubleshooting-app`, `80 → 80`
`broken-pod.yaml`	`project-broken-pod`, image **`nginx:this-tag-does-not-exist`**

13.2 The steps

DO THIS YOURSELF

Lab 13.1 — The full challenge, in her order.

```
cd "Class-15-Kubernetes Troubleshooting/mini-project"

# 1. Deploy
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get pods
kubectl get service

# 2. Check the app — the full ladder on one healthy pod
kubectl get pods -o wide
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl exec -it <pod-name> -- bash
  curl localhost        # nginx welcome page
  exit

# 3–4. Check the Service and its endpoints
kubectl describe service troubleshooting-service     # Selector, TargetPort, Endpoints
kubectl get endpoints troubleshooting-service        # two pod IPs

# 5–6. The broken pod — and you are NOT allowed to touch the YAML yet
kubectl apply -f broken-pod.yaml
kubectl get pod project-broken-pod
kubectl describe pod project-broken-pod              # read Events

# 8. Break the Service yourself: edit service.yaml → selector app: wrong-app
kubectl apply -f service.yaml
kubectl get endpoints troubleshooting-service        # <none>

# 9. Find it and fix it
kubectl get pods --show-labels
kubectl describe service troubleshooting-service
# put the selector back to app: troubleshooting-app, apply, re-check endpoints
```

WATCH OUT

Her rule for step 6: *you are NOT allowed to immediately change the YAML.* You must first prove the cause with `get` → `describe` → Events. The point of the exercise is the investigation, not the fix.

13.3 The five broken-pod questions — model answers

Try them yourself first.

LOCK THIS IN

Model answers — Section 7.

Q	Answer
1. Pod status?	`ErrImagePull`, settling into `ImagePullBackOff`. READY `0/1`, RESTARTS `0`.
2. Actual error?	In Events: `Failed to pull image "nginx:this-tag-does-not-exist"` — the registry reports the manifest for that tag **not found**.
3. Which command found it?	`kubectl describe pod project-broken-pod` — the **Events** section. (`kubectl logs` can't: no container ever started.)
4. What's wrong with the image?	The **tag** `this-tag-does-not-exist` doesn't exist on Docker Hub. The name `nginx` is fine.
5. How to fix?	Change it to a real tag, e.g. `image: nginx:1.27`; then `kubectl delete pod project-broken-pod` and `kubectl apply -f broken-pod.yaml`; verify `1/1 Running`.

13.4 The troubleshooting table — model answer

LOCK THIS IN

Model answer — Section 11.

Problem	What I saw	Command I used	Root cause	Fix
Broken Pod	`0/1 ImagePullBackOff`, 0 restarts	`kubectl get pod` → `kubectl describe pod` (Events)	image tag doesn't exist	real tag `nginx:1.27`, delete + re-apply
Service Problem	Service exists with a ClusterIP, but `ENDPOINTS <none>`	`kubectl get endpoints`, `get pods --show-labels`, `describe service`	selector `app: wrong-app` ≠ pod label `app: troubleshooting-app`	selector back to `app: troubleshooting-app`, re-apply, endpoints show 2 IPs
Image Problem	`Failed to pull image … not found` in Events	`kubectl describe pod`	wrong tag (could also be wrong name / private registry with no pull secret)	correct the `image:` line or add an `imagePullSecret`

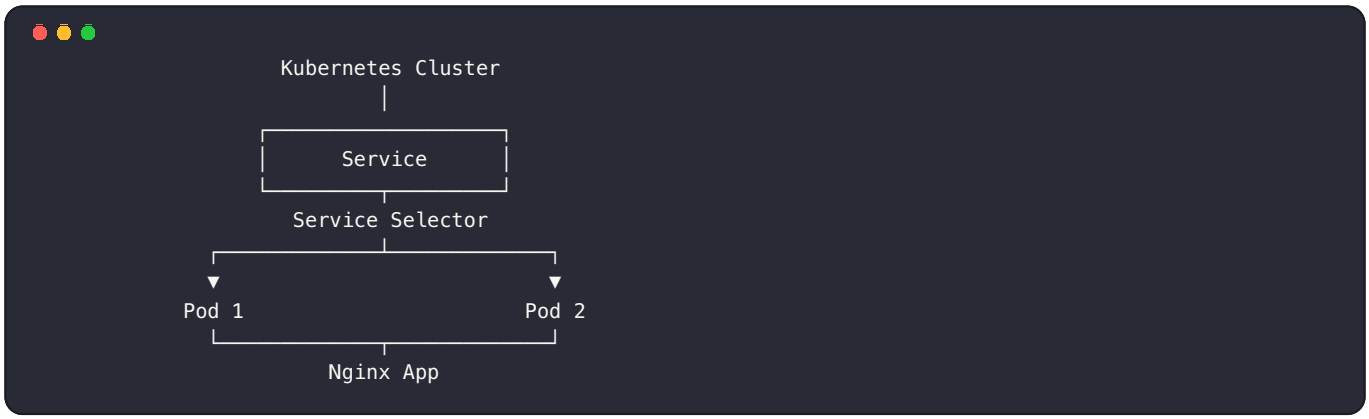
13.5 The ten README questions — model answers

LOCK THIS IN

Model answers — Section 12. Write your own first; these are for checking.

1. **What does `kubectl get` tell us?** The current state of resources — name, READY, STATUS, RESTARTS, AGE. *What* is happening, not *why*.
2. **`get` vs `describe`?** `get` is a one-line summary per object. `describe` is the full detail for one object — containers, state, last state, exit codes, conditions, and the **Events** that explain *why*.
3. **Why `kubectl logs`?** To read what the application wrote to stdout/stderr — its own explanation of what went wrong. `--previous` gets the last crashed container's logs.
4. **When `kubectl exec`?** When the container is running and you need to check from inside — files, config, `curl localhost`, DNS from the pod's point of view. Not useful if the container won't stay up.
5. **`CrashLoopBackOff`?** The container starts but keeps exiting; Kubernetes keeps restarting it, waiting longer between attempts each time. It's a symptom — the cause is in the logs.
6. **`ImagePullBackOff`?** Kubernetes can't pull the image — wrong name, wrong tag, private registry without credentials, or registry/network unreachable — and is backing off before retrying.
7. **Why can a pod stay `Pending`?** The scheduler can't place it: no node matches its nodeSelector/affinity, no node has enough free CPU/RAM for its requests, taints without tolerations, or an unbound PVC.
8. **Why can a Service have no endpoints?** Its selector matches no pod labels — or the matching pods aren't Ready, or there are no pods at all.
9. **Service selector vs pod labels?** The selector is a filter; every pod whose labels match it becomes an endpoint of the Service. They must match exactly — key and value.
10. **What is Kubernetes DNS?** CoreDNS, running in `kube-system`, gives every Service a name — `service.namespace.svc.cluster.local` — so pods can reach each other by name instead of by changing IPs.

13.6 The target architecture



13.7 Her final rule

LOCK THIS IN

When something breaks: DON'T GUESS.

```
GET → DESCRIBE → EVENTS → LOGS → EXEC → TEST → FIX → VERIFY
```

That is the basic Kubernetes troubleshooting mindset.

---

14 🔥 The Triage Gauntlet — Five Broken Pods at Once

Her repo's `scenarios/` folder (not in your copy) goes one step further than the mini-project: **five deliberately broken "production" workloads, all deployed at once**, and you have to triage them. Every pod carries the label `tier: triage-gauntlet` so you can filter them.

```
# scenarios/triage_all.sh — what it does
kubectl apply -f scenario-1-crashloop/broken.yaml
kubectl apply -f scenario-2-imagepull/broken.yaml
kubectl apply -f scenario-3-pending/broken.yaml
kubectl apply -f scenario-4-dns-failure/broken.yaml
kubectl apply -f scenario-5-oomkilled/broken.yaml
sleep 5
kubectl get pods -l tier=triage-gauntlet      # "CURRENT CLUSTER CARNAGE"
```

GOOD TO KNOW

Script detail worth stealing. `set -euo pipefail` makes the script stop at the first failed command, treat unset variables as errors, and fail a pipeline if any part fails. And `SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"` means the script works **no matter which directory you run it from** — every path is relative to the script itself. Both are good habits from the shell-scripting class.

Roughly what you'll see *(filled in — timings vary)*:

```
NAME                     READY   STATUS             RESTARTS
fail-1-crashloop-pod     0/1     CrashLoopBackOff   2
fail-2-imagepull-pod     0/1     ImagePullBackOff   0
fail-3-pending-pod       0/1     Pending            0
fail-4-dns-failure-pod   1/1     Running            0        ← the sneaky one
fail-5-oomkilled-pod     0/1     OOMKilled / CrashLoopBackOff
```

14.1 Scenario 1 — CrashLoop: the missing environment variable

```python
db_url = os.environ.get("DATABASE_URL")
if not db_url:
    print("[FATAL ERROR]: DATABASE_URL environment variable is MISSING!", file=sys.stderr)
    sys.exit(1)
print("Application started successfully!")
```

Symptom	`CrashLoopBackOff`, restarts climbing
Diagnose	`kubectl logs fail-1-crashloop-pod --previous` → `[FATAL ERROR]: DATABASE_URL environment variable is MISSING!`
Root cause	the app requires `DATABASE_URL`; the pod spec has no `env:`
Fix	add `env: [{name: DATABASE_URL, value: "postgres://…"}]` — or, properly, take it from a **Secret** (Class 14)

CLARIFIED

Clarified — the "fix" still restarts. Once `DATABASE_URL` is set, the script prints "*Application started successfully!*" and then **ends** (exit 0). With the default `restartPolicy: Always`, it's restarted, ends again, and you get `Completed` → `CrashLoopBackOff` all over again — just with no error this time. That's the §8 🔍 note in the wild. A real server would keep running; for this demo you'd also need something like `restartPolicy: Never` or a long-running process to see a clean `Running`.

14.2 Scenario 2 — ImagePull: the typo'd private-looking image

```yaml
      # BUG: Typo in image name and tag
      image: yatri-api-service:v999-invalid-tag-does-not-exist
```

Symptom	`ErrImagePull` ⇄ `ImagePullBackOff`, 0 restarts
Diagnose	`kubectl describe pod fail-2-imagepull-pod` → Events: `pull access denied, repository does not exist or may require authorization` *(typical wording)*
Root cause	no registry prefix → Kubernetes looks for `docker.io/library/yatri-api-service`, which doesn't exist; the tag is invalid too
Fix	a real image and tag; for a private registry, the full path (`myregistry.io/team/yatri-api-service:v1`) plus an `imagePullSecret`

14.3 Scenario 3 — Pending: the 500-core pod

```yaml
      resources:
        requests:
          # BUG: Demanding 500 CPU cores ensures the pod can never be scheduled!
          cpu: "500"
          memory: "1000Gi"
```

Symptom	`Pending`, `NODE <none>`
Diagnose	`kubectl describe pod fail-3-pending-pod` → `FailedScheduling … Insufficient cpu, … Insufficient memory`
Root cause	requests 500 cores and 1000 GiB RAM — no node on earth has that free
Fix	realistic requests, e.g. `cpu: "100m"`, `memory: "128Mi"`. Compare against `kubectl describe node` → *Allocatable*

14.4 Scenario 4 — DNS failure: the pod that lies

```
echo "Attempting connection to internal database...";
# BUG: Incorrect internal hostname
curl -s --connect-timeout 3 http://postgres-db-wrong-name.production.svc.cluster.local:5432 || true
echo "Process sleeping...";
sleep 3600
```

Symptom	`1/1 Running`, **0 restarts.** Looks perfectly healthy in `get`.
Diagnose	`kubectl logs` shows only "*Attempting connection…*" and "*Process sleeping…*" — no error. Then: `kubectl exec fail-4-dns-failure-pod -- nslookup postgres-db-wrong-name.production.svc.cluster.local` → can't resolve
Root cause	the hostname points at a Service (and a `production` namespace) that don't exist
Fix	the correct Service name/namespace — check with `kubectl get svc -A`

HARD MODE

Why this is the hardest one. `get` says *Running*. `describe` has no Warning events. `logs` has no error. Three of the five commands come back clean. Two things are hiding the failure: `curl -s` ("silent") suppresses the error message, and `|| true` swallows the non-zero exit code so the script carries on. The only way to catch it is to **test connectivity from inside** — step 8 of her 11-step process. Lesson for your own scripts: use `curl -sS` (silent, but **S**how errors) and don't paper over failures with `|| true`.

GOOD TO KNOW

Filled in — `curlimages/curl` has no `nslookup`. It's a minimal image. If `nslookup` isn't found, run the DNS check from your `dns-test` pod (§12) instead, or `kubectl exec … -- curl -v http://<name>` and read the `Could not resolve host` line.

14.5 Scenario 5 — OOMKilled: the memory leaker

```
      command:
        - "python3"
        - "-c"
        - |
          # BUG: Rapidly allocates 200MB of RAM while limit is 20Mi
          print("Allocating memory rapidly...")
          chunks = []
          for i in range(100):
              chunks.append(b"x" * (10 * 1024 * 1024))
      resources:
        limits:
          memory: "20Mi"
```

Symptom	`OOMKilled`, then `CrashLoopBackOff`
Diagnose	`kubectl describe pod fail-5-oomkilled-pod` → **Last State: Terminated, Reason: OOMKilled, Exit Code: 137**
Root cause	the app tries to hold far more memory than its 20 MiB **limit**
Fix	raise `limits.memory` to what the app genuinely needs — or fix the leak

WATCH IT HAPPEN

What OOMKilled looks like from the kernel's side.

```
Frame 1  python3 starts                          memory ≈ 10 Mi
Frame 2  chunks.append(10 MiB)                   memory ≈ 20 Mi  ← at the limit
Frame 3  chunks.append(10 MiB)                   memory > 20 Mi
Frame 4  Linux OOM killer: SIGKILL (signal 9)    no warning, no cleanup, no error log
Frame 5  Exit Code 137 = 128 + 9                 Reason: OOMKilled
Frame 6  restartPolicy Always → restart → Frame 1 … → CrashLoopBackOff
```

Why the logs are useless: SIGKILL can't be caught, so the app never gets to print "*out of memory*". You'll see "*Allocating memory rapidly...*" and nothing else. The evidence lives in `describe` (Reason + Exit Code), not in logs.

NOTE

A small slip in her comment. It says "*allocates 200MB*", but the loop is `100 × 10 MiB` = **~1000 MiB**. It doesn't change the outcome — it dies the moment it crosses 20 MiB, on roughly the second chunk.

14.6 The gauntlet at a glance

#	Pod	STATUS	The one command that cracks it	Root cause
1	`fail-1-crashloop-pod`	CrashLoopBackOff	`logs --previous`	missing `DATABASE_URL`
2	`fail-2-imagepull-pod`	ImagePullBackOff	`describe` → Events	image name/tag don't exist
3	`fail-3-pending-pod`	Pending	`describe` → FailedScheduling	500 CPU / 1000Gi requested
4	`fail-4-dns-failure-pod`	**Running**	`exec … nslookup`	wrong service hostname
5	`fail-5-oomkilled-pod`	OOMKilled	`describe` → Last State, 137	20Mi memory limit

DO THIS YOURSELF

Lab 14.1 — Run the gauntlet. Clone her repo (`git clone --depth 1 https://github.com/Nency-Ravaliya/devops-heros`), then:

```
cd devops-heros/session-14-kubernetes-troubleshooting/scenarios
bash triage_all.sh
kubectl get pods -l tier=triage-gauntlet -w     # let them settle for a minute
```

Rules: cover up §14.1–14.5. For each pod, write down *status* → *command* → *root cause* → *fix* **before** opening the YAML. Then fix each one (delete → edit → apply) until all five are healthy. Clean up with `kubectl delete pods -l tier=triage-gauntlet` .

15 📂 Your Own Files — A Review

15.1 `deployment.yaml` — `yatri-backend` ✅ mostly very good

```
# condensed — your file uses ordinary block style
metadata:
  name: yatri-backend
  labels: { app: yatri-backend, version: "1.0.0" }
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
  ...
      containers:
        - name: backend
          image: python:3.11-alpine3.19
          command: ["python3", "-c", "import http.server; print('Backend v1.0.0 listening on 5000');
http.server.test(HandlerClass=http.server.SimpleHTTPRequestHandler, port=5000)"]
          ports: [{ containerPort: 5000 }]
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits:   { cpu: "250m", memory: "256Mi" }
```

What's good:

- `maxSurge: 1`, `maxUnavailable: 0` — a true zero-downtime rollout: it always brings up a new pod *before* taking an old one down (Class 12's strategy material, applied).
- `version: "1.0.0"` **is quoted.** Label values must be strings. `1.0.0` would happen to stay a string even unquoted, but `version: 1.0` would become a **number** and the apply would be rejected — so quoting version-like values every time is the right habit.
- **Requests *and* limits are set.** After §10 and §14 you know exactly why: requests let the scheduler place it (no Pending), limits stop one pod from eating the node (and 256Mi is plenty for `http.server` — no OOMKilled).
- **A pinned image tag** (`3.11-alpine3.19`), not `latest` . No ImagePullBackOff surprises from a tag moving under you.

One thing that will bite you in a troubleshooting class specifically:

WATCH OUT

Your startup `print()` may never show up in `kubectl logs`. When Python's stdout isn't a terminal (and inside a container it isn't), Python **buffers** output in a block and only flushes it later. So `kubectl logs yatri-backend-…` might show nothing — not even "*Backend v1.0.0 listening on 5000*" — while the server is happily running. (Request lines from `http.server` go to stderr, which isn't buffered, so those *will* appear once you hit it.)

Fix — either one:

```
command: ["python3", "-u", "-c", "..."]        # -u = unbuffered
# or
env:
  - name: PYTHONUNBUFFERED
    value: "1"
```

It's the kind of thing that sends you chasing a "broken" pod that isn't broken — §5's whole lesson is that logs are the app's voice, so make sure it can be heard.

15.2 `service.yml` — `nginx-service` ⚠️ matches nothing in this folder

```
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

On its own it's a perfectly valid NodePort Service. But nothing in this class folder carries the label `app: nginx` — your Deployment is `app: yatri-backend` on port **5000**, and her pods are `app: web`, `app: troubleshooting-app`, etc. Apply it here and:

```
kubectl get endpoints nginx-service
NAME            ENDPOINTS
nginx-service   <none>
```

…which is *exactly* the §11 failure. If it was meant to front your backend, the fix is two lines:

```
  selector:
    app: yatri-backend      # match the pod label
  ports:
    - port: 80
      targetPort: 5000      # match the containerPort
      nodePort: 30080
```

TRY IT YOURSELF

Turn your own files into a lab. Apply both as they are, confirm `ENDPOINTS <none>`, walk her 7-step checklist from §11.3 to *prove* why, then apply the two-line fix and reach it with `minikube service nginx-service --url` (on macOS, the NodePort isn't reachable at `minikube ip` directly — that's Docker Desktop's VM, as in Class 13).

15.3 The small stuff

File	Note
`Readme.md-2`	Empty. If this was meant for the mini-project write-up, §13.3–13.5 is the shape it needs.
`01-kubectl-get/sample-workload.yaml`	Identical to her `pod.yaml` — just renamed. The README still says `kubectl apply -f pod.yaml`, so use the new name.
`09-…/service.yaml`	Yours has the **working** selector `app: web`; hers now has `app: web-ahsgdf` (§11.1).
`09-…/pod.yaml`	In her repo, not yours — it's a duplicate of the `03-kubectl-logs` pod, unused by that folder's README. Nothing missed.

16 Cheat-Sheet

16.1 The five core commands

Command	Question it answers
`kubectl get pods`	What is happening?
`kubectl describe pod <p>`	What details explain it?
`kubectl logs <p>`	What is the app saying?
`kubectl exec -it <p> -- sh`	What can I see from inside?
`kubectl get events`	What did Kubernetes try, and what happened?

16.2 `get`

Command	Does
`kubectl get pods -o wide`	+ pod IP, node
`kubectl get pods -w`	watch live
`kubectl get pods --show-labels`	show labels
`kubectl get pods -l tier=triage-gauntlet`	filter by label
`kubectl get all`	pods, svc, deploy, rs (not *literally* all)
`kubectl get nodes` / `--show-labels`	node health / node labels
`kubectl get svc` / `deploy` / `endpoints <svc>`	other resources

16.3 `describe`, `logs`, `exec`, `debug`

Command	Does
`kubectl describe pod|deployment|service|node <name>`	full detail + Events
`kubectl logs <p> -f`	follow
`kubectl logs <p> --previous`	the last **crashed** container
`kubectl logs <p> -c <container>`	one container of many
`kubectl logs -n kube-system -l k8s-app=kube-dns`	CoreDNS logs
`kubectl exec -it <p> -- bash`	shell (`sh` on alpine/busybox)
`kubectl exec <p> -- <cmd>`	one-shot command
`kubectl debug -it <p> --image=busybox:1.36`	attach a tool container *(filled in)*

16.4 Events

Command	Does
`kubectl get events --sort-by=.lastTimestamp`	in time order
`kubectl get events --field-selector type=Warning`	problems only
`kubectl events --watch`	live stream
`kubectl events --for pod/<p>`	one object
(default retention)	**1 hour**

16.5 Status → meaning → first move

STATUS	Means	First move	Common causes
`CrashLoopBackOff`	starts, keeps stopping	`logs --previous`	app error, bad command, missing env, bad config, failed dependency, bad probe, permissions, process just exits
`ErrImagePull` / `ImagePullBackOff`	can't get the image	`describe` → Events	wrong name, wrong tag, private registry auth, registry/network down
`Pending` (Node `<none>`)	can't be scheduled	`describe` → `FailedScheduling`	CPU/RAM requests, nodeSelector, affinity, taints, unbound PVC
`OOMKilled` / Exit 137	over memory limit	`describe` → Last State	limit too low, memory leak
`Running` but broken	app/network issue	`exec` → `curl localhost`, `nslookup`	wrong hostname, port, selector

16.6 Service & DNS

Command	Checks
`kubectl get endpoints <svc>`	did the selector find pods? `<none>` = no
`kubectl get endpointslices -l kubernetes.io/service-name=<svc>`	modern equivalent *(filled in)*
`kubectl describe svc <svc>`	Selector, Port, TargetPort, Endpoints
`kubectl exec -it dns-test -- nslookup <svc>`	DNS resolves?
`kubectl exec dns-test -- wget -qO- http://<svc>`	HTTP reaches a pod?
`kubectl exec -it dns-test -- cat /etc/resolv.conf`	which DNS server / search domains
`kubectl get pods -n kube-system`	is CoreDNS running?
FQDN	`service.namespace.svc.cluster.local`

16.7 Exit codes *(filled in)*

Code	Meaning
0	success (still restarted under `restartPolicy: Always`)
1	generic app error
126 / 127	can't execute / command not found
137	SIGKILL — usually **OOMKilled**
143	SIGTERM — asked to stop

16.8 The two rules

```
GET → DESCRIBE → EVENTS → LOGS → EXEC → TEST → FIX → VERIFY
Pod label → Service selector → Endpoints → Service IP → DNS name
```

---

17 🗒️ Notes on the Transcript Itself

The instructor. The transcript header credits **"Ritesh prajapati"**. That's the developer of the *Scaler++ Chrome extension* that downloaded the file — **not** the instructor. This session, like all of them, was taught by **Nensi Ravaliya**.

What was captured: almost nothing. The header says `Lecture Title: Kubernetes 8`, `Start Time: 21/09/2026, 16:15:00`, `Duration: 120 minutes`, `Model use: groq, whisper-large-v3-turbo`. The body is a single paragraph — roughly thirty seconds of a class settling in:

> *"…can we start with the session. Okay, tap the yes, thumbs up, we start with the session. Sharing my screen. Are you able to see my screen? … Now is it visible? Hello. Hello. Can anyone type? Is it visible? Are you able to see my screen? Thank you. Thank you."*

…and then it ends. Either the recording stopped or the extension captured only the opening. **None of the teaching exists in audio form.**

How these notes were built instead. With the user's approval, every technical section comes from her repo folder for this session — `devops-heros/session-14-kubernetes-troubleshooting` — which is what she screen-shared. All YAML, commands, expected outputs, flows and lists of causes are taken from her READMEs and manifests. Your class folder's copy was diffed against her repo: the only differences are the renamed `sample-workload.yaml`, the working-vs-broken `service.yaml` selector, a whitespace change in one README, and the `scenarios/`, `demo-chart/` and `09-…/pod.yaml` files missing from your copy.

What's deliberately absent. No 🎤 quotes, no 😄 class moments, no quiz/team-relay tables, no homework deadlines — none were recorded, and none have been invented. If there were live failures, questions from students or career stories, they're lost.

What's labelled "filled in". Standard reference facts added where her material names a thing without explaining it: the restart back-off timings, exit codes, `kubectl debug`, the EndpointSlices deprecation, typical Events/FailedScheduling wordings, `/etc/resolv.conf` contents, requests vs limits, taints, and the Python stdout-buffering note in §15. The combined decision map in §2.3 is a synthesis of her separate per-problem flows.

Clarifications added (🔍): `CrashLoopBackOff` also happens when a process exits *successfully* under `restartPolicy: Always` (§8.4, §14.1); `Pending` as a `STATUS` vs as a pod **phase** (§10.5).

Small issues found in her files: `02-kubectl-describe/README.md` says `pod.yaml` but the file is `demo-pod.yaml` (§4.3); the OOMKilled comment says 200 MB but the loop asks for ~1000 MiB (§14.5); the top-level README's folder list says `08-pending-pod` and `09-service-dns`, but the real folders are `08-pending-pods` and `09-service-dns-troubleshooting`.

Garbled-term corrections: none needed — there was no technical speech to correct.